컴파일러 퀴즈 답안지 (exam.txt)
연습 1. 다음 설명에 해당하는 컴파일러 단계를 쓰시오.
소스 코드를 한 글자씩 읽으면서, if, while, 숫자 상수, 식별자 같은 토큰(token) 들로
잘라내는 단계.
답: 어휘 분석 (Lexical Analysis)

연습 2. 다음 중 어휘 분석(lexical analysis) 단계에서 사용하는 도구/개념이 아닌것은?
① 정규 표현식
② 토큰(token)
③ 파스 트리(parse tree)
④ 패턴(pattern)
답: ③ 파스 트리(parse tree)

연습 3. 다음 설명에 들어갈 말을 쓰시오.
토큰의 모양(예: 숫자, 식별자 등)을 기술하기 위해 사용하는 형식 언어로, a*, [0-9]+ 같은
식으로 쓴다. 정규____
답: 정규 표현식 (Regular Expression)

연습 4. 다음 중 구문 분석(syntax analysis) 단계에서 하는 일로 가장 알맞은것을
고르시오.
① 토큰을 만들어 낸다.
② 프로그램을 실행해서 결과 값을 계산한다.
③ 토큰의 나열이 문법에 맞는지 검사하고, 파스 트리/AST를 만든다.
④ 실행 시간에 메모리를 할당한다.
답: ③ 토큰의 나열이 문법에 맞는지 검사하고, 파스 트리/AST를 만든다.

연습 5. 다음의 빈칸을 채우시오.
소스 코드에서 괄호, 쉼표 등 표현에만 필요한 기호는 대부분 버리고,
의미상 중요한 구조(연산자, 피연산자, 문장)를 노드로 가지는 트리를
___ ___ 트리라고 부른다.
답: 추상 구문 트리 (Abstract Syntax Tree)

연습 6. 다음 보기 중 추상 구문 트리(AST) 에 해당하는 특징만 모두 고른 것은?
ㄱ. 구체적인 문법 요소(괄호, 세미콜론)를 최대한 유지한다.
ㄴ. 의미상 중요한 노드(연산, 변수, 문장)가 중심이 된다.
ㄷ. 코드 생성과 최적화의 입력으로 사용될 수 있다.
① ㄱ
② ㄴ
③ ㄱ, ㄴ
④ ㄴ, ㄷ
답: ④ ㄴ, ㄷ

연습 7. 다음 설명에 해당하는 용어를 쓰시오.
각 변수 이름과 함수 이름에 대해
타입(type)
스코프(scope)
메모리 위치(offset, 주소 등)
을 저장하는 자료구조.
답: 심볼 테이블 (Symbol Table)

연습 8. 다음 중 심볼 테이블(symbol table) 에 들어갈수있는정보가 아닌 것은?
① 지역 변수의 타입
② 함수의 파라미터 개수
③ 레지스터 할당 결과
④ 전역 변수의 위치 정보
답: ③ 레지스터 할당 결과

연습 9. 소스 코드와 기계어 사이에 두는 중간 표현을 영어 약어 또는 풀네임으로 쓰시오.
답: IR (Intermediate Representation)

연습 10. Bison 파일의 일부이다. 빈칸에 들어갈 키워드를 쓰시오.
```
%{
  /* C 코드 */
%}

/* 토큰 선언 부분 */
%____ NUM
%____ ID
%%
/* 규칙들... */
```
→ 각 빈칸에 들어갈 키워드는?
답: %token NUM, %token ID

연습 11. Bison에서 +, - 연산자가 좌결합(left associativity) 이고,
우선순위도 지정하려고 할 때 쓰는 선언 키워드를 쓰시오.
답: %left

연습 12. 다음 중 Bison에서 연산자 우선순위/결합법칙을 지정할 때 쓰는 것들만 있는 보기를 고르시오.
ㄱ. %left
ㄴ. %right
ㄷ. %token
ㄹ. %nonassoc
① ㄱ, ㄴ
② ㄱ, ㄷ
③ ㄱ, ㄴ, ㄹ
④ ㄷ, ㄹ
답: ③ ㄱ, ㄴ, ㄹ

연습 13. 다음 용어들을 올바른 단계와 연결하시오.
(A) 어휘 분석
(B) 구문 분석
(C) 의미 분석
타입 규칙을 검사한다.
토큰을 만든다.
토큰 나열이 문법에 맞는지 검사하고 트리를 만든다.
→ (A) = __, (B) = __, (C) = __
답: (A) = 토큰을 만든다., (B) = 토큰 나열이 문법에 맞는지 검사하고 트리를 만든다., (C) = 타입 규칙을 검사한다.

연습 14. 다음 중 어휘 분석 단계의 입력과 출력의 올바른 짝을 고르시오.
입력: (ㄱ) 소스 코드 문자 스트림
출력: (ㄴ) 토큰 스트림
① 입력 ㄱ, 출력 ㄴ
② 입력 ㄴ, 출력 ㄱ
③ 입력/출력이 같다
④ 어휘 분석과는 관계없다
답: ① 입력 ㄱ, 출력 ㄴ

연습 15. 빈칸에 들어갈 한글 용어를 쓰시오.
lex / flex 로 구현하는 것은 보통 _____ 분석기이다.
yacc / bison 으로 구현하는 것은 보통 _____ 분석기이다.
답: 어휘, 구문

연습 16. 다음 정의를 보고 해당하는 용어를 쓰시오.
컴파일러에서 front-end의 결과물로 생성되어, back-end에서 최적화하고 기계어를 만드는데 사용되는 중간단계표현.
답: 중간 표현 (Intermediate Representation)

`exam2.txt` 시험 대비 공부용 힌트 문서

1. 어휘 분석 (Lexical Analysis / Scanning)
주요 역할: 소스 코드의 문자 스트림을 읽어서 의미 있는 최소 단위인 '토큰(Token)'으로 분리합니다.
토큰의 구성: 일반적으로 토큰의 '타입(type)'과 '값(value)'으로 구성됩니다.

줄 번호/열 정보의 중요성 (샘플 1 관련)
핵심: `진단 메시지 (Diagnostic message)`
설명: 컴파일러가 오류를 발견했을 때, 사용자가 소스 코드에서 해당 오류의 정확한 위치를 파악하고 수정할 수 있도록 정밀한 정보를 제공하기 위함입니다. 이는 사용자 친화적인 오류 보고에 필수적입니다.

2. 구문 분석 (Syntax Analysis / Parsing)
주요 역할 (샘플 3 관련):
- 어휘 분석 단계에서 생성된 토큰들의 나열이 프로그래밍 언어의 '문법 규칙(Grammar Rules)'에 맞는지 검사합니다.
- 문법에 맞다면, 소스 코드의 구조를 계층적으로 표현하는 '트리 구조(Parse Tree 또는 Abstract Syntax Tree)'를 생성합니다.
오답 노트: 단순히 토큰으로 쪼개는 것은 어휘 분석, 타입/스코프 정보 저장은 의미 분석, 중간 표현을 기계어로 변환하는 것은 코드 생성/최적화 단계의 역할입니다.

3. 의미 분석 (Semantic Analysis)
주요 역할: 구문 분석이 생성한 트리 구조를 바탕으로 프로그램의 '의미적 정확성'을 검사합니다.
예: 변수가 선언되었는지, 타입이 일치하는지, 함수 호출 인자의 개수가 맞는지 등을 확인합니다.
주요 도구 : '심볼 테이블(Symbol Table)'을 활용하여 변수, 함수 등의 정보(타입, 스코프 등)를 관리합니다.

4. 컴파일러 구조의 전반부 (Front-end)
구성 (샘플 2 관련): 어휘 분석, 구문 분석, 의미 분석, 그리고 중간 코드 생성(Intermediate Code Generation) 단계를 통틀어 '프론트엔드(Front-end)'라고 부릅니다.
역할: 소스 언어에 의존적이며, 소스 코드를 중간 표현(IR)으로 변환하는 역할을 담당합니다.

5. 트리 구조의 이해: 파스 트리 vs. 추상 구문 트리 (AST)
파스 트리 (Parse Tree / Concrete Syntax Tree):
특징: 문법 규칙의 모든 세부 사항을 반영하여, 소스 코드의 토큰 및 비단말(non-terminal) 심볼을 포함하는 상세한 트리입니다.

예: 괄호, 세미콜론 등 문법적으로 필요하지만 의미적으로 중요하지 않은 요소들도 포함합니다.

추상 구문 트리 (Abstract Syntax Tree, AST) (샘플 4 관련):
특징: 소스 코드에서 괄호, 쉼표 등 문법적으로 필요한 '불필요한 기호'를 대부분 생략하고, 프로그램의 '의미상 중요한 구조(연산, 피연산자, 문장)'만을 노드로 표현한 간결한 트리입니다.
용도: 컴파일러의 이후 단계(의미 분석, 코드 생성, 최적화)에서 활용하기에 파스 트리보다 훨씬 효율적입니다.
핵심: 추상 구문 트리는 프로그램의 의미를 효과적으로 나타내는 추상화된 표현입니다.

6. 심볼 테이블 (Symbol Table)
역할: 프로그램 내에서 사용되는 모든 식별자(변수, 함수, 타입 등)의 이름과 그에 대한 정보(타입, 스코프, 저장 위치 등)를 저장하고 관리하는 자료구조입니다. 의미 분석 단계에서 필수적으로 사용됩니다.

7. 중간 표현 (Intermediate Representation, IR)
개념: 소스 언어와 타겟 머신 언어 사이에 존재하는 추상적인 코드 형태입니다.
장점: IR을 사용하면 컴파일러의 재활용성이 높아지고, 최적화 단계에서 다양한 변환을 적용하기 용이합니다. (샘플 3, 4의 선택지에 포함되어 중요성 인지)

8. 파서 파일 (Parser Files) (힌트 3 관련)
개념: `Flex` (렉서 생성기)와 `Bison` (파서 생성기, Yacc의 GNU 버전)과 같은 도구를 사용하여 문법 정의(`*.y`) 및 어휘 규칙 정의(`*.l`) 파일로부터 자동으로 생성되는 소스 코드 파일(예: `parser.tab.c`, `lex.yy.c`)을 의미합니다.
역할: 이 파일들은 컴파일러의 어휘 분석기와 구문 분석기를 실제로 구현하는 코드입니다.

9. 레지스터 이름 (힌트 2 관련)
중요성: 코드 생성 단계(백엔드)에서 프로그램 변수나 중간 결과가 CPU 레지스터에 어떻게 할당되는지 이해하는 데 필요합니다.
예시 (x86 아키텍처 기준)**:
-`EAX (Accumulator Register): 산술 연산 결과, 함수 반환 값 등에 주로 사용됩니다.
-`EBX` (Base Register): 데이터 포인터나 오프셋 주소 지정에 사용됩니다.
-`ECX` (Counter Register): 루프 카운터 등에 사용됩니다.
-`EDX` (Data Register): 산술 연산의 확장이나 I/O 포인터 등에 사용됩니다.
-`ESP` (Stack Pointer): 스택의 최상단 주소를 가리킵니다.
- EBP` (Base Pointer / Frame Pointer): 현재 스택 프레임의 베이스 주소를 가리키며, 지역 변수나 함수 인자에 접근하는 데 사용됩니다.
- `EIP` (Instruction Pointer): 다음에 실행될 명령어의 주소를 가리킵니다.

소스 코드 (문자 스트림) ⬇ 어휘 분석 (Lexical Analysis)
토큰 스트림 (Token Stream) ⬇ 구문 분석 (Syntax Analysis)
파스 트리 / AST (Tree 구조) ⬇ 의미 분석 (Semantic Analysis)
중간 표현 (IR) 생성 (Front-end 끝) ⬇ 최적화 및 코드 생성 (Back-end)
타겟 기계어 / 어셈블리

. 단계별 핵심 포인트 (상세)

① 어휘 분석 (Lexical Analysis / Scanning)

하는 일:소스 코드를 한 글자씩 읽어 **의미 있는 조각(Token)**으로 잘라냄.

입력:문자 스트림 (Source Code) $\rightarrow$**출력:**토큰 스트림

도구:정규 표현식 (Regular Expression), Lex/Flex

핵심:

공백, 주석은 제거한다.

진단 메시지(Diagnostic message):에러 발생 시 줄 번호/위치를 알려주기 위해 위치 정보를 추적한다.

② 구문 분석 (Syntax Analysis / Parsing)

하는 일:토큰들이 **문법 규칙(Grammar)**에 맞게 나열되었는지 검사하고 **구조(Tree)**를 만듦.

입력:토큰 스트림 $\rightarrow$**출력:**파스 트리 또는 AST

도구:CFG (Context-Free Grammar), Yacc/Bison

★ 중요 개념: 파스 트리 vs AST

파스 트리 (Parse Tree):문법의 모든 디테일(괄호, 세미콜론 등)을 다 포함. 너무 방대함.

추상 구문 트리 (AST):불필요한 기호는 버리고 **의미상 중요한 연산자, 피연산자**만 남긴 간결한 트리. (컴파일러가 실제로 쓰는 것)

③ 의미 분석 (Semantic Analysis)

하는 일:문법적으로는 맞지만 **의미적으로 말이 되는지**검사.

체크리스트:변수 선언 여부, **타입(Type) 일치 여부**, 함수 인자 개수 확인.

핵심 도구:심볼 테이블 (Symbol Table)

3. 필수 암기: 도구와 키워드 (Bison/Yacc)

퀴즈(연습 10~12)에 나왔던 Bison 문법은 꼭 외우세요.

정의 파일:lex.yy.c(Lex 결과), parser.tab.c(Bison 결과)

Bison 키워드:

%token: 토큰 선언 (예: %token NUM)

%left: 좌결합 (Left Associativity) + 우선순위 지정

%right: 우결합 (Right Associativity)

%nonassoc: 결합 법칙 없음

(참고: %token은 우선순위 지정용이 아님, 선언용임)

---

4. 핵심 자료구조 2대장

A. 심볼 테이블 (Symbol Table)

역할:변수/함수의 **신상명세서**저장소.

저장 정보:이름, **타입(Type)**, **스코프(Scope)**, 메모리 주소(Offset).

*오답 주의:*레지스터 할당 정보나 기계어 코드는 여기에 저장하지 않음.

B. 중간 표현 (IR, Intermediate Representation)

역할: Front-end와 Back-end 사이의 통역사.

이유:소스 언어나 타겟 기계가 바뀌어도 중간 단계(IR)가 있으면 컴파일러 구조를 재활용하고 최적화하기 좋음.

---

5. (보너스) 레지스터 상식 (x86)

백엔드 관련 문제나 힌트에 언급된 레지스터 용도입니다.

EAX:산술 연산 결과, 리턴값 (계산기 화면 같은 존재)

ESP:스택 포인터 (스택의 꼭대기)

EBP:베이스 포인터 (스택 프레임의 기준점, 지역변수 찾을 때 씀)

EIP:다음에 실행할 명령어 주소 (Instruction Pointer)

🏁 시험 직전 1분 요약 (매칭 게임)

1. **Lex / Flex → 어휘 분석 → 정규 표현식**
2. **Yacc / Bison → 구문 분석 → AST 생성**
3. **타입 검사 → 의미 분석 → 심볼 테이블 활용**
4. **Front-end → 중간 표현(IR) 생성**

1. 이론적 배경 (오토마타 연결)

컴파일러 단계는 이론적인 '오토마타' 개념과 짝지어 이해하면 명확합니다.

단계	사용하는 문법	이론적 기계 모델	비고
어휘 분석	**정규 언어**(Regular Grammar)	**유한 오토마타**(DFA/NFA)	정규 표현식으로 표현 가능
구문 분석	**문맥 자유 문법**(CFG)	**푸시다운 오토마타**(Stack 사용)	괄호 짝 맞추기, 중첩 구조 처리 가능

2. Bison/Yacc 심화 (코드 동작 원리)

퀴즈에 Bison 코드가 나왔으니, 이 메커니즘을 이해해야 합니다.

호출 순서:메인 프로그램이 yyparse()(구문 분석기)를 호출합니다.

토큰 요청:yyparse()는 문법을 맞추다가 토큰이 필요하면 yylex()(어휘 분석기)를 호출합니다.

데이터 전달:

yylex()는 토큰의 **타입**(예: NUM)을 리턴합니다.

토큰의 실제 **값**(예: 100, 3.14)은 yylval이라는 전역 변수에 담아서 전달합니다.

[퀴즈 11, 12번 심화] 우선순위와 결합법칙

%left: 1 - 2 - 3을 (1 - 2) - 3으로 해석 (왼쪽 먼저 묶음)

%right: a = b = c를 a = (b = c)로 해석 (오른쪽 먼저 묶음, 대입 연산자 등)

우선순위: Bison 파일의 **아래쪽**에 적을수록 우선순위가 **높습니다**.

코드 스니펫

%left '+' '-' (우선순위 낮음)

%left '*' '/' (우선순위 높음 - 곱하기가 더하기보다 먼저!)

---

3. 구문 분석의 두 가지 방식 (Top-down vs Bottom-up)

Bison은 **Bottom-up**방식을 사용합니다.

Top-down (하향식):큰 문법에서 시작해서 쪼개 나가는 방식. (구현하기 쉽지만 느릴 수 있음)

Bottom-up (상향식):토큰들을 하나씩 읽어서 합쳐 나가며 트리를 만드는 방식. (**Bison/Yacc**이 쓰는 방식, **Shift-Reduce**방식이라고도 함)

---

4. 중간 표현 (IR)의 구체적 형태: 3-주소 코드 (Three-Address Code)

IR이 뭔지 물어볼 때 가장 대표적인 예시가 3-주소 코드입니다.
명령어 하나에 피연산자가 최대 3개(결과 1개, 입력 2개)만 오는 형태입니다.
소스 코드:x = a + b * c
3-주소 코드 (IR) 변환:
Plaintext
t1 = b * c (임시변수 t1 사용)
t2 = a + t1 (임시변수 t2 사용)
x = t2

$\rightarrow$이렇게 쪼개 놓으면 기계어로 바꾸기가 아주 쉬워집니다.

---

5. 최적화 (Optimization) 맛보기

컴파일러 백엔드에서 하는 최적화의 대표적인 예시들입니다. (혹시 예제를 묻는 문제가 나온다면)
상수 접기 (Constant Folding):x = 3 + 4$\rightarrow$컴파일 시간에 x = 7로 미리 계산.
죽은 코드 제거 (Dead Code Elimination):실행될 가능성이 없는 코드(예: if (0) { ... })를 삭제.
공통 부분식 제거:똑같은 계산이 두 번 나오면 한 번만 계산하고 재활용.

---

6. 힌트 파일의 '레지스터' 관련 추가 설명

exam2.txt에 레지스터 설명이 있었죠? 함수가 호출될 때 **스택 프레임**이 어떻게 생기는지 알면 이해가 쉽습니다.
Plaintext
[높은 주소]
  ...
  함수 인자들 (Parameters)
  반환 주소 (Return Address - EIP가 가리키던 곳)
  이전 EBP 값 (Saved EBP) <-- 현재 EBP가 여기를 가리킴
  지역 변수들 (Local Variables)
[낮은 주소 / ESP]

EBP (Base Pointer):"내 방(스택 프레임)의 기준점은 여기야." (그래서 지역변수 찾을 때 EBP - 4, EBP - 8식으로 접근)
ESP (Stack Pointer):"지금 짐(데이터)이 여기까지 쌓였어." (push/pop 할 때마다 움직임)

1. 문법의 모호성 (Ambiguity) - "Dangling Else"

컴파일러 퀴즈의 단골 문제입니다.

상황:if (조건) if (조건) 문장 else 문장

문제:else가 첫 번째 if짝인지, 두 번째 if짝인지 문법적으로 헷갈리는 상황.

해결 (국룰):"가장 가까운 if와 짝짓는다."(Most Closely Nested Rule)

Bison에서의 처리: Shift와 Reduce가 충돌할 때, Bison은 기본적으로 **Shift(더 읽어들이기)**를 선택하여 이 문제를 해결합니다. (즉, else를 안쪽 if에 붙여버림)

2. Bison의 충돌 (Conflict)

Bison을 돌릴 때 경고 메시지로 뜨는 것들인데, 시험에 나오기 딱 좋습니다.

충돌 종류	설명	위험도
Shift/Reduce 충돌	"더 읽어서(Shift) 트리를 키울까, 아니면 지금 묶어서(Reduce) 노드를 만들까?"	보통 Shift를 선택하면 해결됨 (Dangling Else가 대표적). 큰 문제 아님.
Reduce/Reduce 충돌	"규칙 A로 묶을 수도 있고, 규칙 B로 묶을 수도 있는데?"	**심각함.**문법을 잘못 짠 것임. 반드시 고쳐야 함.

3. 정규 표현식 (Regular Expression) 속성 정리

퀴즈 2번, 3번과 관련된 **특수 문자(메타 문자)**의 정확한 의미입니다. 헷갈리지 마세요.

*(Kleene Star): **0번**이상 반복 (없어도 됨) $\rightarrow${0, }

+(Positive Closure): **1번**이상 반복 (최소 하나는 있어야 함) $\rightarrow${1, }

?: **0번 또는 1번**(있거나 없거나) $\rightarrow${0, 1}

|: 또는 (OR)

[]: 문자 클래스 (예: [a-z]는 소문자 중 하나)

[^]: 부정 (예: [^0-9]는 숫자가 **아닌** 것)

. 런타임 환경 (Runtime Environment) - 메모리 관리

힌트 파일의 'Offset', '메모리 위치'와 관련된 내용입니다. 변수가 **어디에**저장되는지 묻는다면 이 3가지입니다.

정적 영역 (Static Area):

전역 변수(Global Variable), static변수.

프로그램 시작부터 끝까지 주소가 고정됨.

스택 영역 (Stack Area):

지역 변수(Local Variable), 함수 인자.

함수가 호출될 때 생겼다가 리턴하면 사라짐. (자동 관리)

힙 영역 (Heap Area):

동적 할당 (malloc, new).

사용자가 직접 관리해야 함. 포인터로 접근.

5. 컴파일러 vs 인터프리터 (기본 상식)

혹시나 해서 짚고 넘어갑니다.

컴파일러:소스 전체를 한 번에 번역 $\rightarrow$실행 파일 생성. (실행 속도 빠름, 번역 시간 걸림)

인터프리터:한 줄씩 읽어서 바로 실행. (실행 속도 느림, 바로 확인 가능)

하이브리드 (Java, C#):소스 $\rightarrow$**중간 코드(Bytecode/IL)**$\rightarrow$VM(가상 머신)이 실행. (이 중간 코드가 아까 배운 IR의 일종입니다.)

어휘 분석기는 문법 오류(괄호 짝 등)를 잡아낼 수 있다? (X- 그건 구문 분석기 몫)

심볼 테이블은 컴파일 전 단계(Front/Back)에서 계속 쓰인다? (O- 정보 조회용으로 계속 씀)

파스 트리는 AST보다 더 간결하고 핵심만 담고 있다? (X- 반대임. AST가 핵심 요약본)

Bison 파일(.y)은 C언어 코드로 변환된다? (Otab.c파일로 변환되어 컴파일됨)

[Step 1] 기초 O/X 및 단답형 (빠르게 훑기)

문제 1.

컴파일러의 첫 번째 단계로, 소스 코드를 읽어 공백과 주석을 제거하고 '토큰(Token)'을 생성하는 단계는?

답:어휘 분석 (Lexical Analysis)

문제 2. (O/X)

구문 분석(Syntax Analysis) 단계에서는 변수의 타입이 올바른지, 선언되지 않은 변수를 사용했는지 검사한다.

답: X

(해설: 타입 검사와 선언 여부 확인은 '의미 분석(Semantic Analysis)' 단계에서 합니다. 구문 분석은 문법 구조만 봅니다.)

문제 3.

다음 중 심볼 테이블(Symbol Table)에 저장되는 정보가 아닌 것은?

① 변수의 이름

② 변수의 스코프(Scope)

③ 변수의 데이터 타입(Type)

④ 변수에 할당된 레지스터 번호

답: ④ 변수에 할당된 레지스터 번호

(해설: 레지스터 할당은 백엔드의 코드 생성 단계에서 결정됩니다. 심볼 테이블은 주로 프론트엔드에서 의미 분석을 위해 씁니다.)

[Step 2] 실전 객관식 & 빈칸 채우기 (시험지 느낌)

문제 4.

다음 설명에 해당하는 트리 구조를 쓰시오.

"파스 트리(Parse Tree)에서 괄호, 세미콜론 등 구문적으로만 필요한 요소를 제거하고, 연산자와 피연산자 등 프로그램의 핵심 의미만을 남겨 간결하게 표현한 트리."

답:추상 구문 트리 (Abstract Syntax Tree, AST)

문제 5.

Bison(Yacc) 설정 파일에서 연산자의 **우선순위(Precedence)**와 **결합 법칙(Associativity)**을 설정하려고 한다. 다음 중 올바른 설명은?

① %token 키워드로 우선순위를 설정한다.

② 파일의 상단에 적을수록 우선순위가 높다.

③ %left는 좌결합, %right는 우결합을 의미한다.

④ %nonassoc은 결합 법칙을 무시하고 강제로 결합한다.

답: ③

(해설: ② 파일의 하단에 적을수록 우선순위가 높습니다. ④ %nonassoc은 결합을 허용하지 않는다는 뜻입니다(예: a < b < c 금지).)

문제 6.

x86 아키텍처의 레지스터 중, 현재 스택 프레임의 **기준 주소(Base Address)**를 가리키며, 함수 내에서 지역 변수나 파라미터에 접근할 때 기준으로 사용되는 레지스터는?

① EAX

② EIP

③ ESP

④ EBP

답: ④ EBP (Base Pointer)

(참고: ESP는 스택의 끝(Top)을 가리키고, EIP는 다음 실행할 명령어를 가리킵니다.)

[Step 3] 고득점 변별력 문제 (헷갈리는 개념)

문제 7.

다음 중 Bison/Yacc에 대한 설명으로 틀린 것은?

① 구문 분석기(Parser)를 생성해주는 도구이다.

② 생성된 파일명은 보통 parser.tab.c 형태이다.

③ 문법 규칙을 정의할 때 정규 표현식(Regular Expression)을 주된 도구로 사용한다.

④ 내부적으로 Shift-Reduce 방식을 사용하는 상향식(Bottom-up) 파서이다.

답: ③

*(해설: 정규 표현식은 **Lex/Flex(어휘 분석)*에서 씁니다. Bison은 CFG(문맥 자유 문법, BNF) 형식을 사용합니다.)

문제 8. (주관식)

컴파일러 프론트엔드가 생성하며, 소스 코드와 기계어 사이에서 다리 역할을 하는 코드를 무엇이라 하는가? (이것이 있으면 타겟 기계가 바뀌어도 프론트엔드를 재사용하기 좋다.)

답:중간 표현 (Intermediate Representation, IR)

문제 9.

if (cond) if (cond) stmt else stmt와 같은 문장에서 else가 어느 if에 속하는지 모호한 문제를 무엇이라 하며, Bison은 이를 어떻게 해결하는가?

답:

용어: **Dangling Else (매달린 else) 문제**

해결: **Shift**를 우선한다. (가장 가까운 if에 else를 붙인다.)